Contents

Day 13 — Testing AI Systems

Traditional software testing is built around a powerful assumption:

> Given the same input and environment, the program should produce the same output.

A simple model is:

```
input
  ↓
deterministic program
  ↓
expected output
```

Testing then becomes straightforward:

```
assert f(x) == expected
```

AI systems change this model.

An LLM may produce multiple valid outputs for the same input:

```
                    +-- Output A
                    |
Input --> Model -----+-- Output B
                    |
                    +-- Output C
                    |
                    +-- Output D
```

The correct abstraction is therefore:

$$P(y \mid x, c, m, s)$$

where:

- x = input
- c = context
- m = model
- s = system state
- y = possible output

Testing is no longer simply:

$$f(x) = y$$

It becomes:

$$P(Y \mid X, C, M, S) \in \text{acceptable behavior}$$

This does **not** mean AI systems cannot be tested rigorously.

It means the testing strategy must evolve from checking exact outputs to checking **properties, distributions, constraints, quality, safety, and system behavior**.

The fundamental transition is:

```
Traditional software

input
  ↓
expected output
  ↓
assert equality


AI systems

input
  ↓
possible outputs
  ↓
evaluate properties
  ↓
measure quality
  ↓
check safety
  ↓
compare against baseline
```

The goal is not to prove that an AI system always produces one exact answer.

The goal is to establish that it **reliably behaves within an acceptable behavioral envelope**.

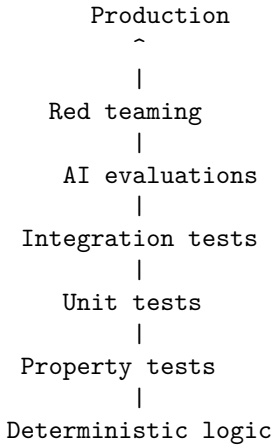
---

1. The Testing Pyramid Still Exists

AI systems do not replace traditional software testing.

They add another layer.

A mature AI application might have:



The bottom of the pyramid should remain heavily tested.

For example:

- authentication
- authorization
- parsing
- schema validation
- token accounting
- retry logic
- caching
- routing
- database operations
- policy enforcement

These should be tested deterministically whenever possible.

The LLM should not be used where a deterministic assertion will do.

---

2. Unit Tests

Unit tests verify isolated components.

For an AI application, many components are still ordinary software.

Examples:

```
chunk_document()
normalize_query()
build_context()
parse_model_output()
validate_tool_arguments()
calculate_cost()
select_model()
enforce_policy()
truncate_context()
```

These should have conventional tests.

For example:

```python
def test_context_does_not_exceed_budget():
    context = build_context(documents, max_tokens=4000)

    assert token_count(context) <= 4000
```

Or:

```python
def test_unauthorized_tool_is_rejected():
    result = authorize(
        user="user_1",
        tool="delete_database",
    )

    assert result == DENY
```

The presence of an LLM somewhere in the architecture does not turn every test into an AI evaluation.

A useful principle is:

> **Test deterministic behavior deterministically.**

---

3. Integration Tests

Unit tests isolate components.

Integration tests verify that components work together.

Consider:

```
User
 ↓
API
 ↓
Retriever
```

```
 ↓
LLM
 ↓
Tool
 ↓
Database
```

An integration test might verify:

```
request
 ↓
retrieval succeeds
 ↓
context constructed correctly
 ↓
LLM receives expected context
 ↓
tool call generated
 ↓
tool authorization succeeds
 ↓
tool executes
 ↓
response returned
```

The exact model output may vary.

The integration contract should therefore focus on structural properties.

For example:

```
+ retrieval was called
+ only authorized documents were retrieved
+ tool call conforms to schema
+ unauthorized tool was rejected
+ final response contains required fields
```

This distinction prevents fragile tests that fail merely because the model phrased something differently.

---

4. Property-Based Testing

Property-based testing asks:

> **What must always be true?**

Instead of enumerating only specific examples, generate many inputs and test invariants.

For example, for a context manager:

$$tokens(context) \leq budget$$

For an authorization system:

$$Unauthorized(action) \Rightarrow Denied(action)$$

For a cost limiter:

$$Cost(execution) \leq Budget$$

For an agent runtime:

$$Steps(execution) \leq MaxSteps$$

These properties are extremely valuable for AI systems because the model introduces enormous input and output variability.

For example:

```
@given(random_tool_arguments())
def test_invalid_arguments_never_execute(args):
    result = execute_tool(args)

    assert result.was_executed is False
```

The model can generate thousands of unusual argument combinations.

The deterministic security layer should reject those that violate its contract.

---

5. Testing the Model Is Different

Suppose the expected answer is:

> "The meeting is scheduled for Tuesday."

The model might produce:

> "The meeting will take place Tuesday."

A string comparison fails.

But the answer is correct.

AI evaluation therefore requires semantic criteria.

For example:

```
Question:
When is the meeting?

Acceptable:
"The meeting is Tuesday."
"Tuesday is the scheduled date."
"The meeting will take place Tuesday."

Unacceptable:
"Wednesday."
"I don't know."
"The meeting is next month."
```

The evaluator is checking a property:

$$Correct(answer, reference) = True$$

rather than exact string equality.

---

6. Evals

An **eval** is a systematic measurement of model or system behavior against a defined criterion.

An evaluation dataset might look like:

```
{
    input,
    context,
    expected_behavior,
    metadata
}
```

The system produces:

`output`

and an evaluator computes:

$$score = E(input, output, expected\ behavior)$$

Examples of evaluation dimensions include:

- correctness
- relevance
- groundedness

- completeness
- citation accuracy
- instruction following
- tool selection
- tool arguments
- safety
- refusal behavior
- latency
- cost

The key insight is:

> **Evals are tests where the oracle is behavioral rather than necessarily exact.**

---

7. The Evaluation Oracle Problem

Traditional testing depends on an oracle:

```
input → expected output
```

AI systems often lack a single exact expected output.

This creates the **oracle problem**.

Suppose the question is:

> Explain why a distributed system needs timeouts.

There may be hundreds of correct answers.

You can therefore evaluate against:

Reference answers

A human-written ideal answer.

Rubrics

A set of required properties.

For example:

```
Must mention:
+ bounded waiting
+ resource exhaustion
+ cascading failure
```

Structured facts

```
required_concepts = {
    "resource exhaustion",
    "bounded latency"
}
```

External verification

For factual questions, compare claims against authoritative sources.

LLM-as-judge

Use another model to evaluate the response.

Each approach has advantages and weaknesses.

A mature evaluation system often combines them.

---

8. LLM-as-Judge

One common technique is to use an LLM to evaluate another LLM.

For example:

```
Question
   ↓
Model under test
   ↓
Answer
   ↓
Evaluator model
   ↓
score + explanation
```

A rubric might ask:

```
Score 1-5:

1. Is the answer correct?
2. Is it grounded in the supplied context?
3. Does it answer the actual question?
4. Does it contain unsupported claims?
```

This is useful because language models can evaluate semantic properties that are difficult to encode with deterministic rules.

But LLM-as-judge has limitations:

- evaluator bias
- model preference
- correlated errors
- sensitivity to phrasing
- difficulty detecting subtle factual errors
- evaluator drift

Therefore:

> **The evaluator is itself a component that must be tested.**

Do not treat an LLM judge as an infallible oracle.

---

9. Pairwise Evaluation

Sometimes absolute scoring is difficult.

Instead, compare two systems:

```
Answer A
   vs.
Answer B
```

and ask:

> Which answer is better?

This produces:

$$A > B$$

or:

$$B > A$$

Pairwise evaluation can be useful for comparing:

- model versions
- prompts
- retrieval strategies
- agent architectures
- routing policies
- context strategies

For example:

```
Version 1 → 100 answers
Version 2 → 100 answers

Human / evaluator comparison
        ↓
Version 2 wins 63%
Version 1 wins 25%
Tie        12%
```

This provides evidence that Version 2 is better, though statistical uncertainty still matters.

---

10. Regression Testing

A regression occurs when a change causes previously working behavior to become worse.

Traditional software regression testing looks like:

```
Code change
   ↓
run tests
   ↓
PASS / FAIL
```

For AI systems:

```
Prompt change
Model change
Retriever change
Tool change
Context change
Routing change
   ↓
run eval suite
   ↓
compare against baseline
```

An AI regression may be subtle.

For example:

```
                        Before      After

Correctness              92%         93%
Groundedness             95%         91%
Latency                  2.1s        1.8s
```

Cost	$0.08	$0.05
Safety	99%	96%

A naive evaluation might conclude:

> "Accuracy improved."

A systems evaluation sees:

> **The release introduced a significant safety regression.**

AI regression testing therefore requires multiple dimensions.

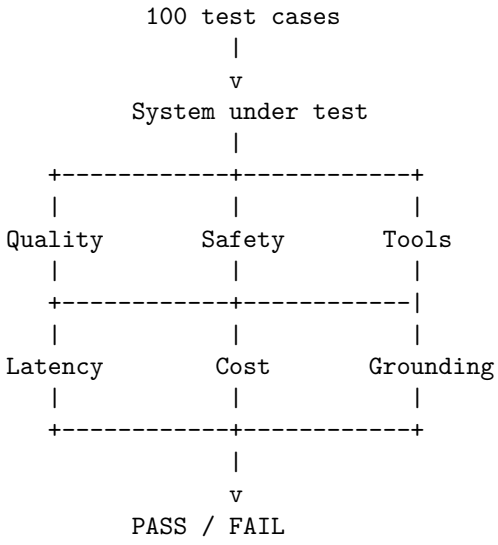
---

11. The AI Regression Suite

This is today's core exercise.

Build a suite containing approximately 100 representative cases.

The pipeline should look like:



Each test case should contain enough information to determine what "good" means.

For example:

```
{
  "id": "research_017",
  "input": "What was the revenue in 2025?",
  "context": ["annual_report.pdf"],
  "expected": {
```

```
    "must_be_grounded": true,
    "must_cite_source": true,
    "expected_fact": "..."
  },
  "limits": {
    "max_latency_ms": 5000,
    "max_cost": 0.10
  }
}
```

The test runner executes the system and records:

```
output
quality
groundedness
safety
tool behavior
latency
tokens
cost
```

---

12. Multi-Dimensional Pass/Fail

A single score can hide important regressions.

Instead, define separate gates.

For example:

```
Quality       >= 90%
Groundedness  >= 95%
Safety        >= 99%
Tool accuracy >= 98%
P95 latency   <= 5 sec
Cost/task     <= $0.10
```

Then:

$$PASS = Q \geq Q_{min} \wedge G \geq G_{min} \wedge S \geq S_{min} \wedge T \geq T_{min} \wedge L \leq L_{max} \wedge C \leq C_{max}$$

This is much stronger than:

```
overall_score = 0.92
```

because a system could score 92% overall while having a catastrophic safety failure.

---

13. Golden Datasets

A regression suite needs representative examples.

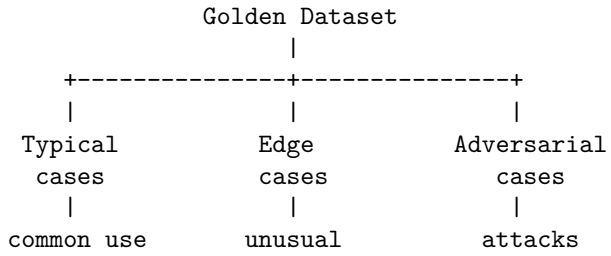
A **golden dataset** is a curated collection of cases whose expected behavior is known and important.

For a research assistant, it might include:

```
factual questions
ambiguous questions
multi-hop questions
unanswerable questions
citation questions
conflicting sources
long-context questions
tool-use questions
security attacks
edge cases
```

The dataset should not consist only of easy examples.

A useful distribution is:



Otherwise the regression suite will systematically overestimate system quality.

---

14. Test Categories

A mature AI test suite should include several classes.

Functional tests

Does the system perform the requested task?

Behavioral tests

Does it behave according to the specification?

Safety tests

Does it refuse or constrain dangerous behavior?

Security tests

Can malicious inputs manipulate the system?

Performance tests

Does it satisfy latency and throughput requirements?

Economic tests

Does it stay within cost budgets?

Tool tests

Does it select and invoke tools correctly?

Retrieval tests

Does it retrieve the right evidence?

Regression tests

Did a change make anything worse?

This gives us:

```
AI Test Coverage
|
+-- functionality
+-- behavior
+-- safety
+-- security
+-- performance
+-- economics
+-- tools
+-- retrieval
+-- regressions
```

---

15. Adversarial Testing

Normal tests ask:

> Does the system work?

Adversarial tests ask:

> How can I make the system fail?

Examples:

```
contradictory instructions
malicious documents
prompt injection
very long inputs
empty inputs
garbled inputs
ambiguous requests
unexpected Unicode
malformed tool results
incorrect tool outputs
extreme token lengths
```

For an agent:

```
What happens if the tool lies?
What happens if the model lies?
What happens if retrieval lies?
What happens if the user lies?
What happens if all three happen together?
```

Adversarial testing is particularly important because AI systems operate on inputs whose semantic space is enormous.

---

16. Fuzzing

Traditional fuzzing generates large numbers of unusual inputs.

For example:

```
random bytes
boundary values
very long strings
malformed JSON
unexpected encodings
```

AI systems can also be fuzzed semantically.

Generate:

```
instruction variations
contradictory prompts
nested instructions
Unicode transformations
prompt injection variants
tool argument mutations
context perturbations
```

For example, if the invariant is:

> Unauthorized tools must never execute.

Generate thousands of variations attempting to convince the model to invoke the tool.

The deterministic authorization layer should continue to enforce:

$$Unauthorized(tool) \Rightarrow DENY$$

This is an important distinction:

> **Fuzz the model; enforce invariants outside the model.**

---

17. Metamorphic Testing

AI systems are particularly suitable for **metamorphic testing**.

Instead of knowing the exact expected output, define transformations that should preserve or predictably change behavior.

Suppose:

```
Input:
"What is the capital of France?"
```

Transform it into:

```
"What is France's capital?"
```

The answer should remain semantically equivalent.

Another example:

```
Original:
Summarize this document.

Transformed:
Please summarize this document.
```

The semantic result should be substantially unchanged.

This creates a metamorphic relation:

$$f(T(x)) \approx f(x)$$

where (T) is a transformation that should preserve the relevant semantics.

Other useful transformations include:

- paraphrasing
- whitespace changes
- reordered irrelevant context
- case changes
- equivalent formatting
- irrelevant distractors

Metamorphic tests are powerful because they do not require a single exact oracle.

---

18. Testing RAG Systems

RAG requires testing at multiple layers.

```
Question
   ↓
Query transformation
   ↓
Retrieval
   ↓
Ranking
   ↓
Context construction
   ↓
Generation
   ↓
Citation
```

Each stage can fail.

Retrieval tests

Measure:

Recall@k

Precision@k

Context tests

Verify:

```
+ relevant documents included
+ irrelevant documents excluded
+ source metadata preserved
+ context within budget
```

Generation tests

Measure:

```
+ answer correctness
+ groundedness
+ citation accuracy
+ unsupported claims
```

A RAG system is therefore not tested simply by asking:

> "Did the answer look good?"

You need to determine **where in the pipeline the failure occurred**.

---

19. Testing Tool Use

Agentic systems require another testing dimension:

> **Did the agent do the right thing?**

Suppose the user asks:

> "Find the latest report and summarize it."

Expected behavior might be:

```
search()
read_document()
summarize()
```

A bad agent might:

```
delete_document()
send_email()
search_private_database()
```

even if the final answer looks reasonable.

Therefore test:

Tool selection

Was the correct tool selected?

Tool arguments

Were the arguments correct?

Tool ordering

Were dependencies respected?

Authorization

Was the action permitted?

Recovery

What happened when the tool failed?

Side effects

Did anything unintended happen?

A tool-call trace is often more informative than the final answer.

---

20. Trace-Based Evaluation

For agents, the final output is only part of the behavior.

Consider:

```
User
 ↓
Planner
 ↓
Search
 ↓
Retrieve
 ↓
LLM
 ↓
Tool
```

```
 ↓
LLM
 ↓
Answer
```

Capture the complete trace:

```
{
    step,
    model,
    input_tokens,
    output_tokens,
    tool,
    arguments,
    result,
    latency,
    cost
}
```

Then evaluate both:

```
Final result
```

and:

```
Execution trajectory
```

This is important because two agents can produce the same answer:

```
Agent A:
3 correct steps


Agent B:
17 steps
2 failed tools
1 unauthorized attempt
```

The outputs may be identical.

The systems are not equally reliable.

---

21. Testing Non-Determinism

Running the same test once is often insufficient.

Suppose a test passes:

```
Run 1 → PASS
```

That does not establish reliability.

Run it repeatedly:

```
Run 1 → PASS
Run 2 → PASS
Run 3 → FAIL
Run 4 → PASS
Run 5 → PASS
```

The observed success rate is:

$$\hat{p} = \frac{4}{5} = 80\%$$

The important question becomes:

> What probability of failure are we willing to tolerate?

For stochastic systems, evaluation should often measure:

$$P(success)$$

rather than simply:

$$success \in 0, 1$$

This is one of the most fundamental differences between deterministic software testing and AI evaluation.

---

22. Statistical Thinking

Suppose a model passes:

```
95 / 100
```

tests.

It is tempting to say:

> "The model is 95% accurate."

That conclusion may be unjustified.

The 100 cases are only a sample.

The result depends on:

- dataset composition

- sampling
- evaluator accuracy
- task distribution
- confidence intervals

For a binomial estimate:

$$\hat{p} = \frac{k}{n}$$

but uncertainty around $\hat{p}$ matters.

If you test:

```
99 / 100
```

you still do not know whether the true rate is 99%.

You know only that the observed sample produced that result.

As the stakes increase, evaluation should therefore incorporate statistical uncertainty.

---

23. Evaluation Drift

The system can change without your code changing.

Examples:

```
model provider changes model
embedding model changes
retrieval index changes
documents change
tool API changes
system prompt changes
```

This means the regression suite should run continuously.

The system should detect:

```
behavioral drift
```

as well as software regressions.

For example:

```
Monday:
Groundedness = 97%

Wednesday:
Groundedness = 92%
```

If the application code did not change, investigate:

- model version
- retrieval corpus
- provider behavior
- evaluator behavior
- infrastructure changes

AI systems need **behavioral observability**.

---

24. Evaluation Data Is a Product

The quality of the evaluation dataset strongly determines the quality of the engineering process.

A weak dataset produces false confidence.

A strong dataset contains:

```
common cases
edge cases
failure cases
historical incidents
security attacks
user complaints
production examples
newly discovered failures
```

When a production failure occurs:

```
production incident
      ↓
reproduce
      ↓
add to regression suite
      ↓
fix system
      ↓
verify regression is gone
```

This creates a feedback loop:

```
Production
   ↓
Failure
   ↓
Test case
   ↓
```

```
Regression suite
   ↓
Engineering change
   ↓
Production
```

Over time, the test suite becomes a **compressed representation of the system's historical failures**.

That makes it one of the most valuable assets in the project.

---

25. Release Gates

The regression suite should become part of the deployment pipeline.

For example:

```
Developer change
      ↓
Unit tests
      ↓
Integration tests
      ↓
AI regression suite
      ↓
Security tests
      ↓
Performance tests
      ↓
Cost checks
      ↓
PASS
      ↓
Deploy
```

A release should fail automatically if critical metrics regress.

For example:

```
Quality        93% → 94%     +
Groundedness   97% → 96%     +
Safety         99.8% → 97.2% x
P95 latency    4.1 → 4.4 s   +
Cost/task      $0.07 → $0.06 +
```

The release is blocked because safety crossed its threshold.

This turns evaluation into an engineering control rather than an analytics dashboard.

---

26. The AI Regression Dashboard

A useful regression report might look like:

```
AI Regression Suite
----------------------------------------

Tests                         100
Passed                         94
Failed                          6

Quality                       93.4%
Groundedness                  96.1%
Safety                        99.2%
Tool accuracy                 97.8%

P50 latency                   2.1 s
P95 latency                   4.7 s
P99 latency                   8.9 s

Avg input tokens              4,820
Avg output tokens               640
Avg cost/task                $0.071

Previous release:
Quality                       94.1%
Groundedness                  96.4%
Safety                        99.5%
P95 latency                   4.3 s
Cost/task                     $0.074

STATUS: FAIL
Reason: Safety regression
```

This is much more informative than:

```
Tests: 94/100
```

---

27. Testing the Test System

There is a subtle but critical problem.

The evaluation infrastructure can itself be wrong.

For example:

```
System output
    ↓
Evaluator
    ↓
PASS
```

What if the evaluator incorrectly labels hallucinations as correct?

Then:

```
Model quality appears high
```

while:

```
Actual quality is low
```

Therefore test the evaluation system.

Use:

- human-reviewed evaluation samples
- evaluator calibration sets
- multiple evaluators
- deterministic checks where possible
- adversarial examples
- known failure cases

The evaluation harness is part of the production engineering system.

It deserves testing like any other component.

---

28. Testing the Entire Agent

Now combine everything.

Consider the Personal Research Assistant:

```
User
 ↓
Intent analysis
 ↓
Retrieval
 ↓
```

```
Context construction
 ↓
LLM
 ↓
Tool calls
 ↓
Validation
 ↓
Response
```

Build test cases covering:

Correct behavior

```
known answer
known document
known tool
```

Uncertainty

```
answer not in corpus
```

Retrieval failures

```
no relevant documents
```

Tool failures

```
timeout
malformed response
permission denied
```

Security

```
prompt injection
malicious document
secret extraction
```

Performance

```
large context
high concurrency
long generation
```

Economics

```
excessive model calls
expensive model routing
cache misses
```

The test system should determine not just whether the answer is correct, but whether the **whole execution was acceptable**.

---

29. The Testing Matrix

A useful way to organize AI testing is as a matrix.

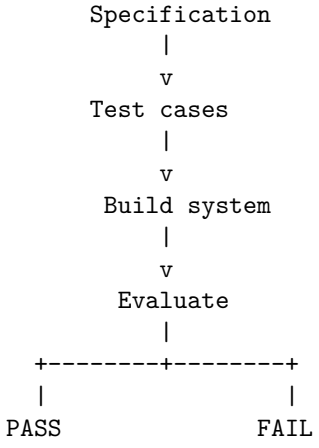
Dimension	Example metric	Example failure
Functional	task success	wrong answer
Quality	correctness	factual error
Grounding	citation correctness	unsupported claim
Retrieval	Recall@k	relevant document missed
Tools	tool accuracy	wrong tool
Safety	unsafe-action rate	prohibited action
Security	attack success	prompt injection
Reliability	failure recovery	timeout cascade
Performance	P95 latency	request too slow
Economics	cost/task	budget exceeded
Robustness	perturbation success	paraphrase changes answer
Regression	delta vs baseline	quality degradation

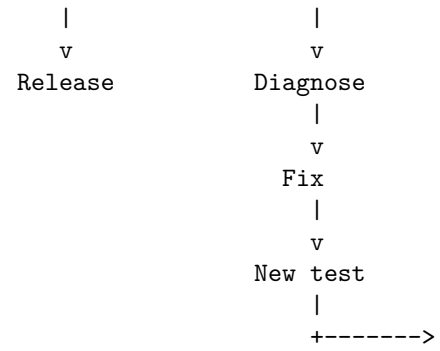
This prevents the common mistake of defining AI quality using one metric.

---

30. Testing as a Feedback Loop

The mature AI engineering process becomes:





---

31. From Tests to Evals

There is sometimes confusion between testing and evaluation.

A useful distinction is:

Test

Usually asks:

> Does this implementation satisfy a specific invariant or contract?

Examples:

```
Unauthorized tool → DENY
Context <= 8,000 tokens
JSON conforms to schema
Timeout terminates request
```

Eval

Usually asks:

> How well does the system perform a behavioral task?

Examples:

```
Answer correctness = 93%
Groundedness = 96%
```

```
Summarization quality = 4.4/5
Tool selection accuracy = 98%
```

Red team

Asks:

> How can I make the system violate its intended behavior?

All three are necessary.

```
Tests
  +
Evals
  +
Adversarial testing
  =
AI assurance
```

---

32. The Testing Mindset

The naive question is:

> "Does the program work?"

The AI engineer asks:

> "What does 'work' mean?"

Then:

> "What properties must always hold?"

Then:

> "What behavior is acceptable even when the output varies?"

Then:

> "How do we detect regressions?"

Then:

> "How do we know our evaluator is correct?"

Then:

> "How does the system behave on adversarial inputs?"

Then:

> "What happens when the model, retrieval system, tool, and evaluator all behave unexpectedly?"

And finally:

> **"How do we know that the system remains within its specified behavioral envelope after every change?"**

That is the central problem of AI testing.

The objective is not to eliminate uncertainty.

It is to **measure and control it**.

Key Takeaways

1. **AI testing is different because outputs are distributions, not fixed values.** The correct abstraction is behavioral acceptance rather than exact string equality.
2. **Traditional testing remains essential.** Deterministic components should continue to use unit, integration, property-based, and conventional regression tests.
3. **Test deterministic behavior deterministically.** Authentication, authorization, parsing, policy enforcement, token limits, cost accounting, and routing should not depend on an LLM judge.
4. **Evals measure behavioral quality.** Correctness, groundedness, relevance, completeness, safety, tool use, and other semantic properties require evaluation rather than simple assertions.
5. **The oracle problem is fundamental.** When many outputs can be correct, use rubrics, references, structured facts, external verification, human evaluation, or carefully validated LLM judges.
6. **LLM-as-judge is useful but imperfect.** The evaluator itself can be biased or wrong and must therefore be calibrated and tested.
7. **Regression testing is mandatory.** Model changes, prompts, retrieval systems, tools, context strategies, and routing policies can all change system behavior.
8. **Build a golden dataset.** Include normal cases, edge cases, adversarial cases, historical failures, security attacks, and real production incidents.
9. **Measure multiple dimensions.** At minimum:

```
quality
safety
grounding
tool accuracy
```

```
latency
cost
```

10. **Do not hide failures behind one aggregate score.** A small improvement in accuracy does not justify a large safety regression.
11. **Property-based and metamorphic testing are especially valuable for AI systems.** Test invariants and semantic relationships rather than only exact outputs.
12. **Agent traces matter.** Evaluate not only the final answer but also tool selection, arguments, authorization, execution trajectory, retries, cost, and side effects.
13. **Test stochastic behavior statistically.** One successful run does not establish reliability; measure success probability and uncertainty where appropriate.
14. **Fuzz the model; enforce invariants outside it.** Generate adversarial and unusual inputs while relying on deterministic security and runtime controls to maintain hard guarantees.
15. **Every production failure should become a test.** This creates a feedback loop:

```
incident
   ↓
reproduce
   ↓
regression case
   ↓
fix
   ↓
permanent protection
```

16. **The evaluation harness is itself production infrastructure.** If the evaluator is wrong, the entire engineering process can acquire false confidence.
17. **Automate the regression suite in CI/CD.** Every meaningful change should automatically measure quality, safety, latency, cost, retrieval, and tool behavior.
18. **Testing becomes part of specification engineering.** The test suite defines what acceptable system behavior actually means.
19. **The central question is not "Does the model work?"**

It is:

> **"Does the entire system remain within its specified behavioral envelope as the implementation changes?"**

For deterministic software, testing asks whether the implementation produces the expected result.

For AI systems, testing asks something more ambitious:

$$\boxed{P(\text{acceptable behavior} \mid \text{input, context, model, state}) \geq \text{required threshold}}$$

That shift—from **exact-output verification** to **behavioral assurance**—is one of the defining changes in AI engineering.